

SentinelAI — Real-Time Cyber Defense Command Center

SentinelAI is an interactive cyber defense platform designed for live simulation, detection, response, and adversary analysis. It combines a cinematic SOC-style frontend with a FastAPI backend that streams realistic attack telemetry, decision outputs, and honeypot behavior in real time.

1) Project Idea

Modern security dashboards are often static, fragmented, and difficult to use in live incident demos.

SentinelAI is built to solve that by providing a **single control plane** where you can:

- launch realistic attack scenarios,
- observe system-level impact across banking infrastructure,
- watch AI-guided response outputs,
- and analyze attacker behavior inside a live honeypot stream.

The platform is optimized for hackathon/demo environments but designed with production-style discipline (validation, tests, CI gates, readiness checks, request tracing).

2) Why SentinelAI Is Unique

- **Unified simulation + SOC UX:** Most projects do either simulation or visualization. SentinelAI does both deeply.
 - **Always-on honeypot flow:** High/critical attack conditions automatically engage honeypot behavior streams.
 - **Multi-layer telemetry:** Pipeline detections, scenario events, infrastructure metrics, side-channel anomalies, and honeypot analysis are streamed together.
 - **Narrative-grade incident replay:** Attack timelines, forensic terminal output, behavior panels, and exportable reports.
 - **Demo reliability tooling:** One-click demo reset, robust reconnect behavior, and route prefetch/lazy loading for smooth live presentations.
-

3) Core Capabilities

Detection & Decision Pipeline

- Simulates attack events (`ddos`, `brute_force`, `sql_injection`, etc.).
- Runs detection and scoring logic to derive threat type, severity, and risk score.
- Produces response actions and copilot explanation output.
- Broadcasts canonical pipeline payloads over WebSocket for dashboard consumption.

Attack Orchestration

- `POST /attack` schedules scripted multi-stage scenarios.
- Emits escalating `scenario_event` frames (`info` -> `warning` -> `high` -> `critical`).
- Injects attack pressure into banking systems over time.
- Triggers honeypot sequence streams in parallel.

Banking Infrastructure Simulation

- Simulates core systems:
 - `api_gateway`
 - `auth_service`
 - `transaction_service`
 - `database`
- Emits continuous `system_update` telemetry:
 - CPU, RPS, latency, error rate, status, anomalies, process list
- Supports attack injection and lateral pressure behavior.

Honeypot Analysis

- Streams adversary behavior as:
 - `honeypot_activity`
 - `honeypot_analysis`
- Shows attacker behavior timeline, pattern detection, and fake terminal transcript.
- Uses the same attack vector catalog as Attack Scenarios UI.

AI Copilot

- Supports local mock mode and A4F OpenAI-compatible provider mode.
- Provides concise SOC guidance from live context.

Reporting

- Live incident summary with timeline.

- Executive metrics (MTTD, containment timing, open criticals, mitigations).
 - JSON and CSV export.
-

4) High-Level Architecture

Frontend (`sentinel-ai-frontend`)

- **Stack:** React + Vite + Tailwind + Framer Motion
- **Realtime State:** `useSyncExternalStore`-style centralized realtime store
- **Realtime Transport:** WebSocket client (`/ws/live`)
- **Pages:**
 - Dashboard
 - Attack Scenarios
 - Systems
 - Infrastructure
 - Honeypot
 - Reports
- **Performance:**
 - lazy-loaded heavy routes
 - sidebar hover/focus route prefetch

Backend (`sentinel-ai-backend`)

- **Stack:** FastAPI + asyncio + Pydantic
 - **Modules:**
 - `engine/` (simulation, detection, decision, response)
 - `services/` (pipeline orchestration, banking simulator, attack orchestrator, honeypot simulator, websocket manager, copilot)
 - `api/routes.py` + `services/attack_orchestrator.py` routers
 - **Operational Additions:**
 - `/api/v1/health` and `/api/v1/ready`
 - request ID middleware (`x-request-id`) + duration logging
 - WS frame schema validation before broadcast
-

5) Realtime Event Model

SentinelAI emits structured payloads over `/ws/live`:

- **Pipeline frame:** includes `event`, `threat`, `actions`, `response`, `explanation`
- **Infrastructure frame:** `system_update`
- **Scenario frame:** `scenario_event`
- **Honeypot frames:**

- `honeypot_activity`
- `honeypot_analysis`
- **Side-channel frames:**
 - `anomaly`
 - `process_log`
 - `recovery`

Backend validates frame contracts before broadcasting. Frontend applies run-time validators before state updates.

6) API Surface

System

- `GET /api/v1/health` — liveness check
- `GET /api/v1/ready` — readiness check (simulator/orchestrator readiness)
- `POST /api/v1/demo/reset` — cancel scenarios + clear banking attacks

Pipeline / Copilot

- `POST /api/v1/pipeline/run` — run one pipeline tick (also broadcasts)
- `POST /api/v1/copilot/chat` — copilot chat response

Banking Simulator

- `POST /api/v1/banking/attack` — inject specific banking attack
- `POST /api/v1/banking/clear` — clear active banking attacks
- `GET /api/v1/banking/snapshot` — one-shot system snapshot

Scenario Orchestrator

- `POST /attack` — trigger attack scenario
- `GET /attack` — list active runs
- `DELETE /attack` — cancel all active runs

Realtime

- `WS /ws/live` — realtime unified stream
-

7) Running the Project

One-command dev run

From repo root:

```
./run-dev.sh
```

This starts backend + frontend together with clean shutdown handling.

Manual run (optional)

Backend

```
cd sentinel-ai-backend
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Frontend

```
cd sentinel-ai-frontend
npm install
npm run dev
```

8) Environment Configuration

Key backend variables (see `sentinel-ai-backend/.env.example`):

- COPILOT_PROVIDER (mock or a4f)
 - A4F_API_KEY
 - A4F_BASE_URL
 - A4F_MODEL
 - simulator/orchestrator interval and attack duration knobs
-

9) Quality, Testing, and CI

Frontend

- ESLint enforced
- Vitest tests
- Coverage reports + threshold gate
- Production build verification

Backend

- Compile checks
- Pytest suites:
 - /attack lifecycle
 - websocket contracts
 - cancel + reconnect behavior

- Coverage gate via `pytest-cov`

GitHub Actions

- Frontend quality job: install, lint, test, coverage, build
 - Backend quality job: install, compile, smoke import, pytest
-

10) Recommended Demo Flow

1. Open dashboard and systems/infrastructure pages.
 2. Trigger a scenario from Attack Control or Attack Scenarios.
 3. Show:
 - threat feed + risk meter spike on Dashboard,
 - infrastructure status shifts and malicious processes,
 - honeypot engagement and analysis stream,
 - reports export.
 4. Use **Demo Reset** to return to clean state instantly.
-

11) Security & Reliability Notes

- Input validation and typed models across API and WS flows.
 - Request correlation via request IDs.
 - Readiness endpoint for deployments and health checks.
 - Defensive client-side WS payload guards.
-

12) Current Scope & Future Enhancements

Current Scope

- Rich simulation and SOC demo platform
- Real-time eventing and visualization
- Basic operational hardening for demo/prototype production readiness

Logical Next Enhancements

- Persistent event storage + replay APIs
 - RBAC/authn/authz for multi-user SOC usage
 - Alert routing integrations (Slack, PagerDuty, email)
 - Full observability stack (Sentry + tracing backend)
 - Extended scenario generator coverage for all frontend vectors
-

13) Repository Structure

```
hack4good/  
  run-dev.sh  
  README.md  
  .github/workflows/ci.yml  
  sentinel-ai-frontend/  
    src/  
    package.json  
    vite.config.js  
    vitest.config.js  
  sentinel-ai-backend/  
    app/  
      api/  
      core/  
      engine/  
      models/  
      services/  
    requirements.txt  
    pytest.ini
```

14) Extended Docs

- docs/ARCHITECTURE.md — deep technical architecture and data flow
 - docs/API_REFERENCE.md — endpoint and websocket reference with examples
 - docs/JUDGE_PITCH.md — judge/demo narrative and talking points
-

15) Vision Statement

SentinelAI is not just a dashboard; it is a **live cyber storytelling engine** for defenders: detect, decide, respond, and investigate in one coherent experience.